Master Guard

https://github.com/baileym12/master-guard

Detecting Unauthorized System Modifications

The Threat of Unauthorized Modifications

Security Breaches

Attackers modify configuration files, binaries, and scripts after gaining system access to maintain persistence or escalate privileges.

Detection Delays

Without active monitoring, these malicious changes can go unnoticed for weeks or months.

Existing Tool Complexity

Current file integrity monitoring tools are highly complex, resource-heavy, and difficult to configure for smaller systems.

The Need

System administrators need a simple, reliable way to quickly identify when files are changed, added, or deleted without permission.

Project Goals & Scope

Develop Master Guard, a lightweight, open-source file integrity monitoring tool built for Linux.

01

Core Functionality

Detect unauthorized changes to critical system directories

02

Simplify Intrusion Detection

Demonstrate core host-based intrusion detection concepts in a practical, easy to use CLI

03

Target Environment

Designed to run on an Ubuntu Linux virtual machine.

Key Features of Master Guard

Baseline Creation

Generates a trusted snapshot of user-specified directories.

Comprehensive Detection

Actively detects:

1. Modified files
2. Newly added files
3. Deleted files

High-Fidelity Tracking

Uses SHA-256 hashes to guarantee accuracy in detecting file alterations.

Manual Approvals

Allows administrators to approve legitimate changes without triggering false positives.

Automation

Easy automated scanning and log generation using cron jobs.

How Master Guard Works

Initialize *(init)*

Scans selected paths and writes file metadata and SHA-256 hashes to a local JSON baseline file.

Monitor *(scan)*

Compares the current state of the file system against the trusted baseline snapshot. Flags any change between the computed hashes and the stored hashes.

Update *(approve)*

If a system administrator performs an expected update, they can update the baseline to reflect the new, trusted system state.

Architecture & Baseline Formatting

Language

Python 3.10+

Using native libraries

Baseline

Stored locally as a lightweight JSON document.

JSON Schema

version:
Tracks the baseline format version.

created_at:
Timestamp of when the snapshot was generated.

paths:
The directories being monitored.

files:
A mapping of the absolute file path to its SHA-256 hex digest.

Modular Code Design

Built with a modular structure for easy maintenance and open-source contributions

src/master_guard/

- cli.py: *Handles all command-line routing and user input.*
- hashing.py: *Contains helper functions for generating SHA-256 digests.*
- scanner.py: *Houses the core logic for building snapshots and comparing file states.*
- storage.py: *Manages reading from and writing to the JSON baseline.*
- models.py: *Defines the data models for the scan reports.*

Setup & Execution

01

Install via Virtual Environment

```
$ python -m venv .venv
$ source .venv/bin/activate
$ pip install -e .
```

02

Creating a Baseline

```
$ master-guard init --paths /etc /usr/bin --baseline baseline.json
```

03

Running a Scan

```
$ master-guard scan --baseline baseline.json
```

04

Approving Changes

```
$ master-guard approve --baseline baseline.json
```

Continuous Monitoring with Cron

Scheduling

1. Open the Cron Editor

```
$ sudo crontab -e
```

2. Add Cron Job *(example from README)*

```
$ */15 * * * * /bin/bash -lc 'source /<path>/.venv/bin/activate && master-guard scan --baseline /<path>/baseline.json >> /var/log/scan.log 2>&1'
```

3. Save & Verify it is Active

```
$ sudo crontab -l
```

Automated Alerting with Exit Codes

0: System is secure *(no changes detected)*.

1: Alert! *(Changes detected: added, modified, or deleted)*.

2: System Error *(e.g., missing permissions)*.

References

Python argparse (CLI design)
https://docs.python.org/3/library/argparse.html

Python hashlib (SHA-256 hashing)
https://docs.python.org/3/library/hashlib.html

Python pathlib (filesystem paths and traversal)
https://docs.python.org/3/library/pathlib.html

Python os module (filesystem operations)
https://docs.python.org/3/library/os.html

Python json module (baseline storage format)
https://docs.python.org/3/library/json.html

Python logging (output/log files)
https://docs.python.org/3/library/logging.html

Python subprocess (integration with system tools)
https://docs.python.org/3/library/subprocess.html

Python stat module (file metadata/permissions checks)
https://docs.python.org/3/library/stat.html

Python hmac.compare_digest (hash comparison)
https://docs.python.org/3/library/hmac.html#hmac.compare_digest

Linux file permissions (ownership, modes)
https://man7.org/linux/man-pages/man1/chmod.1.html

Linux inotify (event-driven file change monitoring)
https://man7.org/linux/man-pages/man7/inotify.7.html

cron reference (alternative scheduler)
https://man7.org/linux/man-pages/man5/crontab.5.html